

# Calculating the Expected Value Function of a Two-Stage Stochastic Optimization Program with a Quantum Algorithm

C. Rotello P. Graf M. Reynolds E. B. Jones C. J. Winkleblack W. Jones

National Renewable Energy Laboratory (NREL) — NatLabRockies

Paper: [arXiv:2402.15029](https://arxiv.org/abs/2402.15029) [quant-ph]

Code: [github.com/NatLabRockies/quantum\\_stochastic\\_programming](https://github.com/NatLabRockies/quantum_stochastic_programming)

## 1 Two-Stage Stochastic Unit Commitment

**Setting:** First-stage gas commitment *before* wind is known; second-stage dispatch *after*.

**Objective** – minimise total expected cost:

$$\min_x \left[ \underbrace{c_g^T x}_{\text{gas cost}} + \underbrace{\mathbb{E}_\xi \left[ \min_{y,r} \{ c_w^T y + c_r^T r \} \right]}_{\phi(x) \text{ (expected value function)}} \right]$$

$x \in \{0, 1\}^{n_g}$  commit gas;  $y \in \{0, 1\}^{n_y}$  dispatch turbines;  $r \geq 0$  recourse;  $\xi \sim p(\xi)$  wind scenarios.

**Challenge:** Computing  $\phi(x)$  over  $2^{n_y}$  scenarios is **#P-Hard** classically. Classical Monte Carlo needs  $O(1/\varepsilon^2)$  samples for error  $\varepsilon$ .

| Method                | Sample complexity    | Space            |
|-----------------------|----------------------|------------------|
| Classical Monte Carlo | $O(1/\varepsilon^2)$ | $O( \Omega )$    |
| Quantum DQA + QAE     | $O(1/\varepsilon)$   | $O(n_y + n_\xi)$ |

## 2 Digitized Quantum Annealing (DQA)

DQA encodes **all wind scenarios simultaneously** in a quantum register and anneals the dispatch register to the per-scenario optimal solution.

$$U_{\text{DQA}} = \prod_{t=1}^T e^{-i\gamma_t H_Q} \cdot e^{-i\beta_t H_M}, \quad H_Q = \sum_{\omega} H_{q,\omega} \otimes |\xi_\omega\rangle\langle\xi_\omega|$$

After  $T$  steps the state approximates the **per-scenario optimized wavefunction**:

$$|\psi_x^*\rangle \approx \sum_{\omega} \sqrt{p(\omega)} |\xi_\omega\rangle |y_\omega^*\rangle$$

Annealing time  $T \sim O(2^{n_y})$  depends only on  $n_y$ , not on  $|\Omega| = 2^{n_\xi}$ .

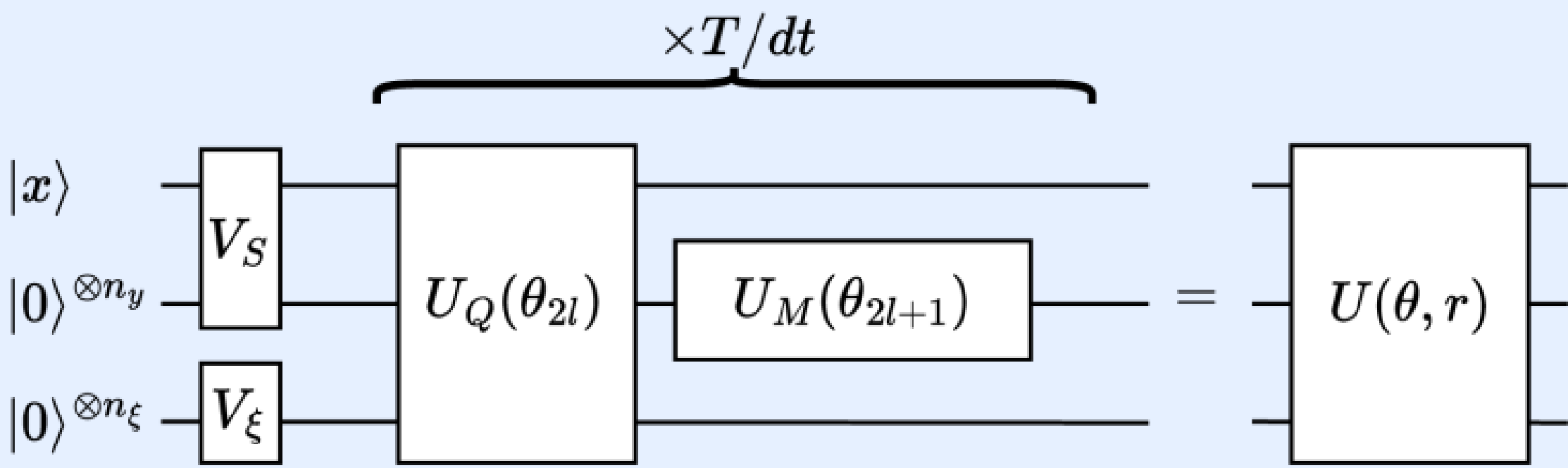


Fig. 1 [1]: DQA circuit. Distribution register  $|\xi\rangle$  acts as classical control for  $U_Q$ .

## 3 Quantum Amplitude Estimation (QAE)

QAE wraps DQA in a Quantum Phase Estimation circuit to extract  $\phi(x)$  with  $O(1/\varepsilon)$  circuit calls.

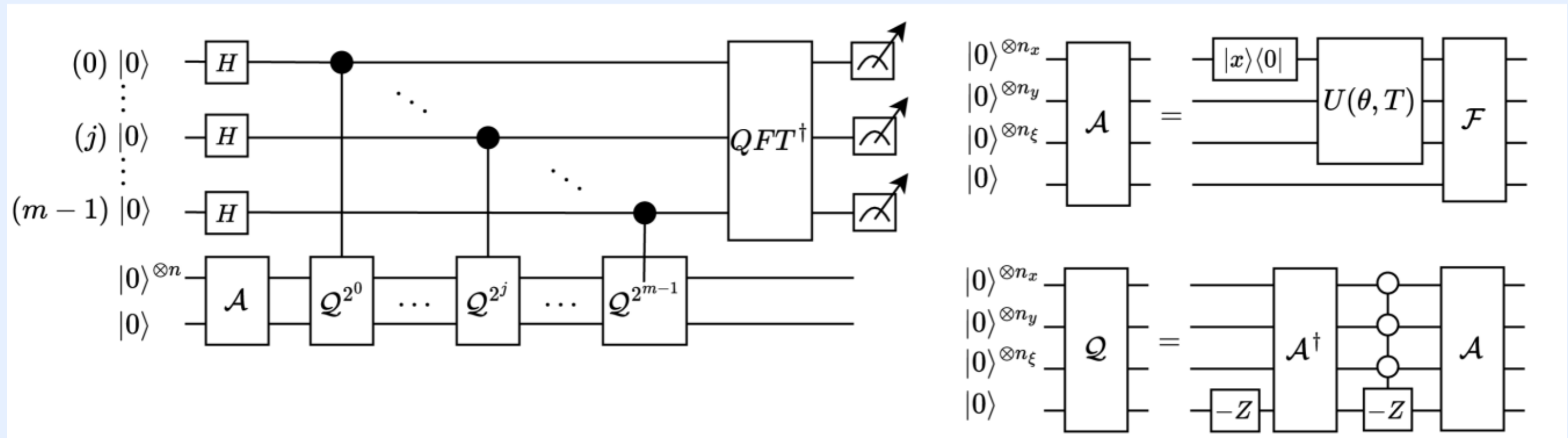


Fig. 2 [1]: QAE circuit.  $A = F \cdot U(\theta, T)$ ;  $Q$  is the Grover iterate.

Error bound [1]: two additive sources – DQA residual  $\delta \sim O(1/T^2)$  and QAE phase estimation  $O(1/M)$ :

$$\Pr \left[ \left| \tilde{\phi}(x) - \phi(x) - \delta \right| \leq \frac{\pi}{M} + \frac{\pi^2}{M^2} \right] \geq \frac{8}{\pi^2}$$

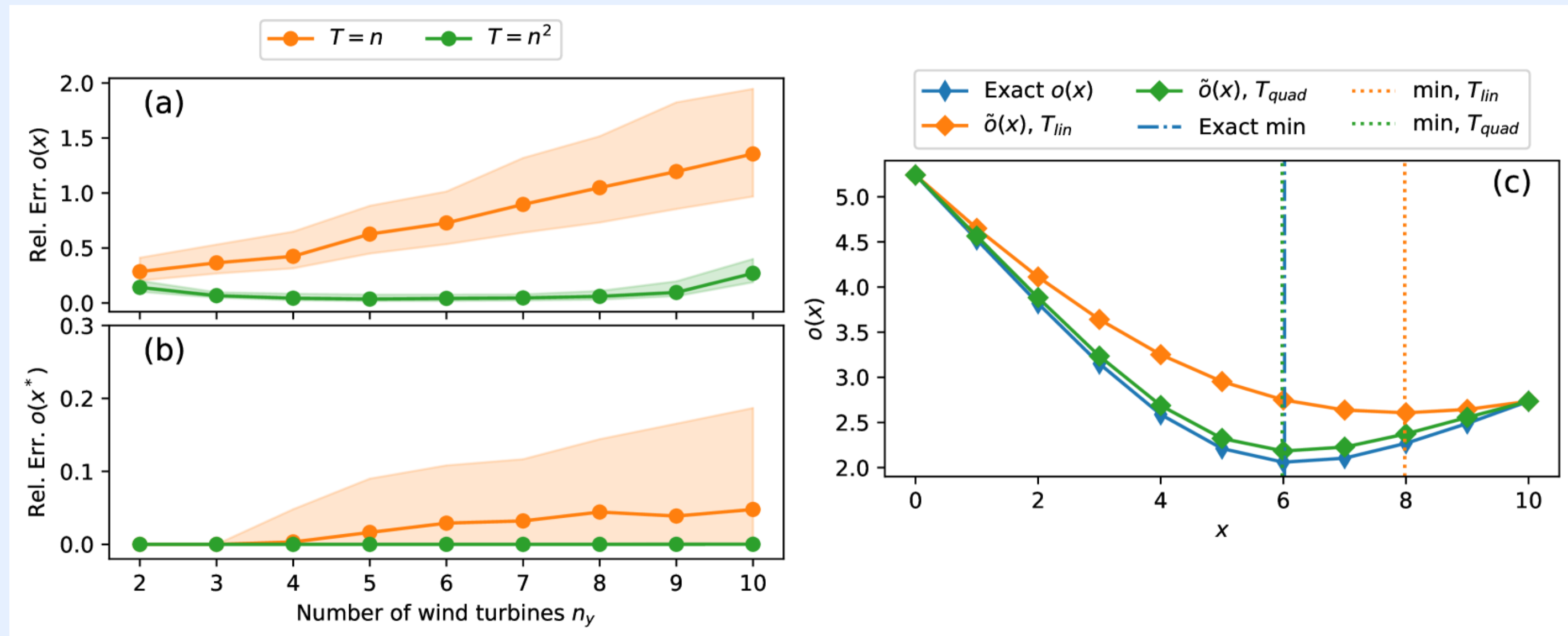


Fig. 3 [1]: DQA convergence.  $T = n_y^2$  gives accurate minima.

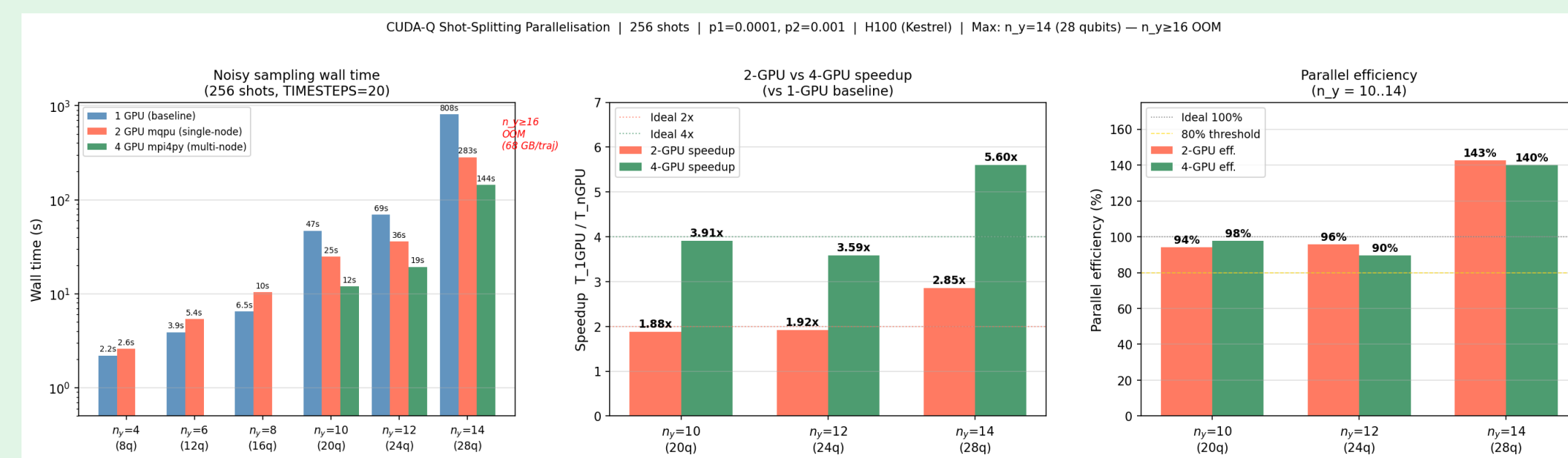
## 4 Three GPU Parallelisation Strategies

Noisy trajectory simulation holds **one full statevector per shot** on-device ( $2^n \times 16$  bytes). Three strategies target this bottleneck on NREL Kestrel H100s (80 GB each).

| Strategy | Mechanism                                                                                                  | Best for                                       |
|----------|------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| mqpu     | Each GPU holds a full statevector; shots dispatched via <code>sample_async(qpu_id=i)</code> (CUDA-Q)       | $n_y \geq 10$ , single node; minimal setup     |
| mpi4py   | One MPI rank per GPU; independent shots, counts gathered via <code>MPI_Gather</code> (shot-split)          | $n_y \geq 10$ , multi-node; near-ideal speedup |
| mgpu     | Statevector <i>sharded</i> across GPUs via <code>cuStateVec</code> + Noiseless ( <code>cuStateVec</code> ) | $n > 33$ qubits                                |

## 5 Noisy Benchmark: mqpu vs mpi4py (Kestrel H100s)

Depolarising noise ( $p_1 = 10^{-4}$ ,  $p_2 = 10^{-3}$ ), 256 shots,  $T = 20$  DQA steps.

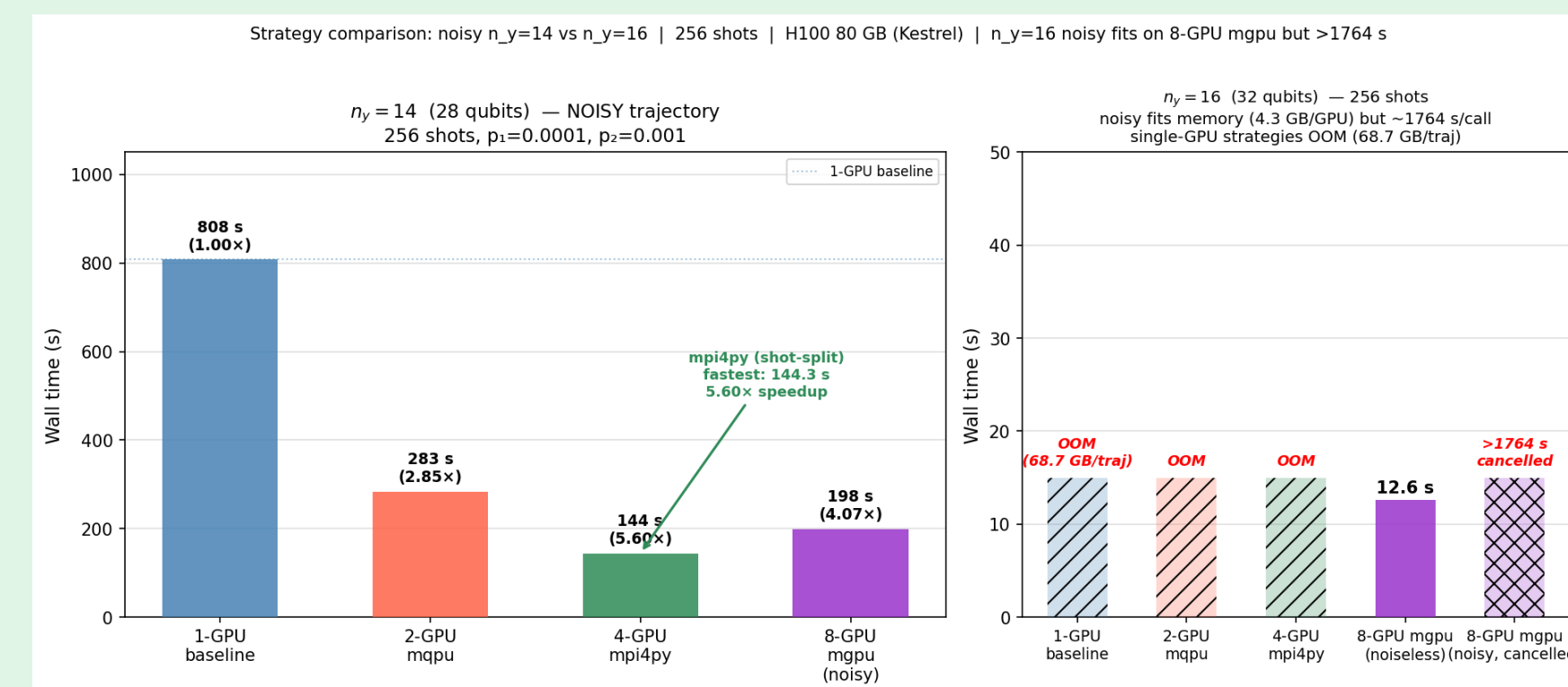


**Left:** Wall time vs  $n_y$  (log scale). mqpu overhead dominates for  $n_y \leq 8$ ; near-2 $\times$  for  $n_y \geq 10$ .

**Centre:** Speedup at  $n_y = 10$ –14: 4-GPU mpi4py reaches 3.6 $\times$ –5.6 $\times$ .

**Right:** Parallel efficiency > 90% for mpi4py at  $n_y = 10$ –12.

## 6 Strategy Comparison at Scale



**Left ( $n_y = 14$ , noisy):** mpi4py 4-GPU = 144 s (5.6 $\times$  vs 1-GPU 808 s). mgpu noisy 8-GPU = 198 s (inter-GPU comms overhead).

**Right ( $n_y = 16$ , 32 q):** Single-GPU strategies OOM (68.7 GB / traj). 8-GPU mgpu noiseless: **12.6 s**. Noisy mgpu fits memory (4.3 GB/GPU) but > 1764 s (cancelled).

## Key Theoretical Findings

- Polynomial speedup:** QAE estimates  $\phi(x)$  with error  $O(1/\varepsilon)$  circuit calls vs  $O(1/\varepsilon^2)$  for classical Monte Carlo — a provable polynomial advantage assuming efficient distribution preparation.
- Parallel scenario optimisation:** Encoding  $p(\xi)$  as a quantum wavefunction and using it as control logic for the phase-separation unitary  $H_Q$  lets DQA converge to the optimal dispatch  $y_\omega^*$  for *all*  $2^{n_\xi}$  scenarios simultaneously in a single quantum circuit.
- Controllable error:** Two independent error sources can each be made arbitrarily small — DQA residual  $\delta \sim O(1/T^2)$  via annealing time  $T$ , and QPE error  $O(\pi/M)$  via register size  $M$ . Setting  $T = n_y^2$  suffices empirically.
- Complexity separation:** Runtime scales as  $O(2^{n_y})$  in annealing time (second-stage turbines) but is *independent* of the number of scenarios  $|\Omega| = 2^{n_\xi}$ , breaking the classical  $O(|\Omega|)$  bottleneck.
- Power grid demonstration:** Algorithm validated on a stochastic unit commitment problem (gas commitment under wind uncertainty), confirming that DQA+QAE recovers the classical expected cost with controllable accuracy.